

Menu.jsx Test Report

Test 1: Category Button Click Updates the Category State

Objective: Verify that clicking a category button updates the category state correctly.

Steps:

1. Render the Menu component with a menuType of "food".
2. Click on the "Starter" button.
3. Wait for the menu title to be displayed.
4. Check if the "Starter" button has the "active" class.

```
test.only('category button click updates the category state', async () => {
  render(<Menu menuType="food" menuTitle="Menu Title" menuDescription="Delicious Food" />);

  const starterButton = screen.getByText(/Starter/i);
  fireEvent.click(starterButton);

  await waitFor(() => {
    expect(screen.getByText(/Menu Title/i)).toBeInTheDocument();
  });

  expect(starterButton).toHaveClass('active');
});
```

Findings: All assertions passed. The category state updates correctly when a category button is clicked.

```
PASS tests/Menu.test.js
  ✓ category button click updates the category state (62 ms)
```

Test 2: Button Works When Clicked

Objective: Ensure the "Add to Cart" button functions correctly.

Steps:

1. Mock API responses for user info and menu items.
2. Render the Menu component with a menuType of "main".
3. Verify that user information ("Welcome, John Doe") appears.
4. Ensure menu items load correctly.
5. Click the "Add to Cart" button.
6. Check if the alert message "Item added to cart" is displayed.

```

test('Button works when clicked', async () => {
  jest.spyOn(window, "alert").mockImplementation(() => { });

  const mockUserInfo = {
    userID: 1,
    firstName: 'John',
    lastName: 'Doe',
  };

  const mockMenuItems = [
    {
      itemID: 101,
      itemName: 'Grilled Chicken',
      itemDescription: 'Delicious grilled chicken',
      itemPrice: 10.99,
      itemImageURL: '/images/chicken.jpg',
    },
  ];

  global.fetch = jest.fn((url, options) => {
    if (url === '/user-info-all') {
      return Promise.resolve({
        ok: true,
        json: () => Promise.resolve(mockUserInfo),
      });
    }
    if (url === '/menu/main?itemType=starter') {
      return Promise.resolve({
        ok: true,
        json: () => Promise.resolve(mockMenuItems),
      });
    }
    if (url === '/add-to-cart' && options?.method === 'POST') {
      return Promise.resolve({
        ok: true,
        json: () => Promise.resolve({ success: true }),
        status: 200,
      });
    }
  });

```

Findings: All assertions passed. The "Add to Cart" button functions as expected.

PASS tests/Menu.test.js

✓ Button works when clicked (137 ms)

Test 3: Active Class is Added to Clicked Category Button

Objective: Verify that clicking a category button adds the "active" class and removes it from other buttons.

Steps:

1. Render the Menu component.
2. Click the "Main" button.
3. Ensure the "Main" button has the "active" class.
4. Verify that "Starter" and "Dessert" buttons do not have the "active" class.

```
test('active class is added to clicked category button', () => {
  render(<Menu menuType="food" menuTitle="Menu Title" menuDescription="Delicious Food" />);

  const starterButton = screen.getByText(/Starter/i);
  const mainButton = screen.getByText(/Main/i);
  const dessertButton = screen.getByText(/Dessert/i);

  fireEvent.click(mainButton);
  expect(mainButton).toHaveClass('active');
  expect(starterButton).not.toHaveClass('active');
  expect(dessertButton).not.toHaveClass('active');
});
```

Findings: All assertions passed. The active class updates correctly when clicking a category button.

```
PASS tests/Menu.test.js
  ✓ active class is added to clicked category button (66 ms)
```

Test 4: Category Switching Triggers Correct API Calls

Objective: Ensure switching categories makes the correct API requests.

Steps:

1. Render the Menu component.
2. Confirm that an API call is made for the default category ("Starter").
3. Click the "Main" button and verify that the API request updates to /menu/food?itemType=main.
4. Click the "Dessert" button and verify the API request updates to /menu/food?itemType=dessert.

```
test.only('category switching triggers correct API calls', async () => {
  render(<Menu menuType="food" menuTitle="Menu Title" menuDescription="Delicious Food" />);

  const starterButton = screen.getByText('Starter');
  const mainButton = screen.getByText('Main');
  const dessertButton = screen.getByText('Dessert');

  // Verify that the initial category (starter) is loaded first
  await waitFor(() => expect(fetch).toHaveBeenCalledWith('/menu/food?itemType=starter'));

  // Click 'Main' and verify API call
  fireEvent.click(mainButton);
  await waitFor(() => expect(fetch).toHaveBeenCalledWith('/menu/food?itemType=main'));

  // Click 'Dessert' and verify API call
  fireEvent.click(dessertButton);
  await waitFor(() => expect(fetch).toHaveBeenCalledWith('/menu/food?itemType=dessert'));
});
```

Findings: All assertions passed. API calls are correctly triggered when switching categories.

PASS tests/Menu.test.js

✓ category switching triggers correct API calls (74 ms)

Test 5: Handles API Failure Gracefully

Objective: Verify that API failures are handled properly without crashing the application.

Steps:

1. Mock an API failure by making fetch return a rejected promise.
2. Render the Menu component.
3. Wait for an error message in the console indicating a failed API request.

```

test('handles API failure gracefully', async () => {
  // Mock console.error to suppress warnings in test output
  jest.spyOn(console, 'error').mockImplementation(() => {});

  // Ensure fetch is mocked
  jest.spyOn(global, 'fetch').mockImplementationOnce(() =>
    Promise.reject(new Error('API failed'))
  );

  render(<Menu menuType="main" menuTitle="Main Menu" menuDescription="Our main dishes" />, {
    wrapper: MemoryRouter,
  });

  await waitFor(() => {
    expect(console.error).toHaveBeenCalledWith(
      expect.stringContaining('Error fetching user info:'),
      expect.any(Error)
    );
  });

  // Restore mocks after test
  console.error.mockRestore();
  global.fetch.mockRestore();
});

```

Findings: All assertions passed. The application correctly logs an error message when the API request fails, ensuring stability.

```

PASS tests/Menu.test.js
  ✓ handles API failure gracefully (70 ms)

```